Milestone 2: Supervisory Baseline

Oct 28, 2026 · @Kevin Berry

Goal

Milestone 2 records what my sorter range looks like when nothing is wrong. Milestone 3 attacks the range, Milestone 4 writes detections, and Milestone 5 hardens it. Each of those needs a measured picture of normal to compare against, or I can't show that an attack changed anything, and a detection can't tell routine polling from an intrusion.

This milestone delivers that picture for one operating state, healthy idle: every service running, the PLC in run mode, no packages moving. I captured one hour of traffic on the PLC's link and the firewall's Level 2 link, then broke it down by conduit: who talks to whom, which Modbus functions they use, how often, and how big the messages are.

Scope change

My Milestone 1 plan put several hours of capture across all sorting states into this milestone. This report delivers one verified hour of a single state, and here is why.

For two days before the due date, the `drives` VM could not run. Its root filesystem was 100% full because the six VFD emulator logs had no rotation and had grown to about 667 MB each. Without `drives`, the range has no motors and no scanners. On October 28 I fixed it by mounting the VM's disk offline from the host, exporting all six logs to evidence with verified SHA-256 hashes, and truncating them in place. The VM then booted with 3.3 GB free.

With the time left, I scoped Milestone 2 to a verified one-hour healthy-idle baseline. This is a reduction from my original plan, not a replacement for it. The remaining baseline work is outstanding and scheduled before the Milestone 3 attacks that depend on it:

- The other operating states: normal sorting, recirculation, outbound backpressure, drive stop/resume, and stopped recovery
- The OPC UA HMI subscription and XLe-to-ASX traffic, which run on loopback inside the guests and need a separate guest-side capture
- Log rotation on the VFD emulators, so the disk cannot fill during longer captures

Range state and preparation

Before capturing, I brought the range up by hand and verified each step.

1. Booted the VMs: `drives`, `fw`, `plc`, `xle`, `scada`, and `analyst`.
2. Started the VFD emulators on `drives`, which had been disabled during the outage. Their logs grew about 10 KB per minute each, roughly 3.6 MB an hour for all six, so the disk was safe for the capture.
3. Started the PLC runtime on `plc`: launched the OpenPLC core, then started its Modbus server on port 502 through the core's local control port. The core answered OK and port 502 began listening.
4. Ran the no-package handshake from `scada`. It returned PASS: the epoch and plant faults cleared from 1 to 0, the PLC opened run epoch 49 with a live plant heartbeat, master off, slots empty, counters zero.
5. Ran the full live preflight. It passed: 43 source and guest file hashes, PLC program identity 24115, all VMs, and every service, including the six VFDs, three scanners, OpenPLC, the OPC UA server, the HMI, XLe, and ASX.

That put the range in healthy idle: everything running and talking, nothing sorting. These steps, the healthy-idle state, and the disk headroom are operator-reported from my terminal session. The packet analysis below does not independently verify them.

Capture method

I captured from two taps on the host. No capture software ran inside the guests.

Tap	Host interface	Attached to	Filter
`fw-l2`	`vnet3`	Firewall's Level 2 interface	TCP ports 502, 4840, 8000
`plc-l1`	`vnet5`	PLC's Level 1 interface	TCP port 502 to or from 10.10.1.10

The capture config listed these taps as `vnet1` and `vnet4`. Libvirt hands out vnet numbers in VM start order, and I had booted the VMs in a different order. I checked each tap by MAC address before capturing and corrected both. With the old mapping, the "PLC" capture would have recorded the firewall's Level 3 interface for the whole hour.

The capture tool caps one run at 600 seconds and 200,000 packets per tap. My first test run hit the packet cap in 193 seconds, because `plc-l1` carries about 1,040 packets per second even at idle. So I ran 24 back-to-back segments of 150 seconds, about 156,000 packets each, under the cap with room to spare. A script ran them in order and would have stopped on the first failure. All 24 completed.

Each segment is sealed with its own SHA-256 manifest. I ran the tool's verifier as root on all 24 sealed originals and each returned COMPLETE, but that output was not saved as a receipt. On the readable copy used for analysis, all 48 pcap hashes matched their manifests and all 24 segments verified COMPLETE. The 24 segments total about 3,603 seconds of capture inside a 60-minute, 34-second window, 17:25:55 to 18:26:29 UTC. There is a gap of about 1.3 seconds between segments, so this is segmented observation, not continuous.

Results

Healthy idle is not quiet. Four independent polling loops (XLe, plant-profile traffic, the PLC, and SCADA) run the whole hour, all Modbus TCP on port 502, with zero reported kernel drops, zero Modbus exceptions, and zero TCP resets observed on either tap. Interface-drop statistics were unavailable.

Conduit	Function codes	Registers	Rate	Poll interval
XLe (10.10.2.20) to PLC	FC1 read coils	Coils 914 to 915, the XLe mode flags	19.82 requests/s	50.45 ms
Plant-profile traffic from `drives` (10.10.1.21) to PLC	FC16 write, FC1 read, FC3 read	Writes 590; reads coils 918 to 921 and registers 509, 558 to 559	9.92 cycles/s	100.82 ms
PLC to each VFD (10.10.1.21 to 26)	FC16 write, FC3 read	Writes 0 to 2 (command, speed reference, belt load); reads 0 to 8	9.50 cycles/s each	105.23 ms
PLC to each scanner (10.10.1.27 to 29)	FC16 write, FC3 read	Writes 0 to 3 (trigger sequence, serial, seed, no-read rate); reads 0 to 10	9.50 cycles/s each	105.23 ms

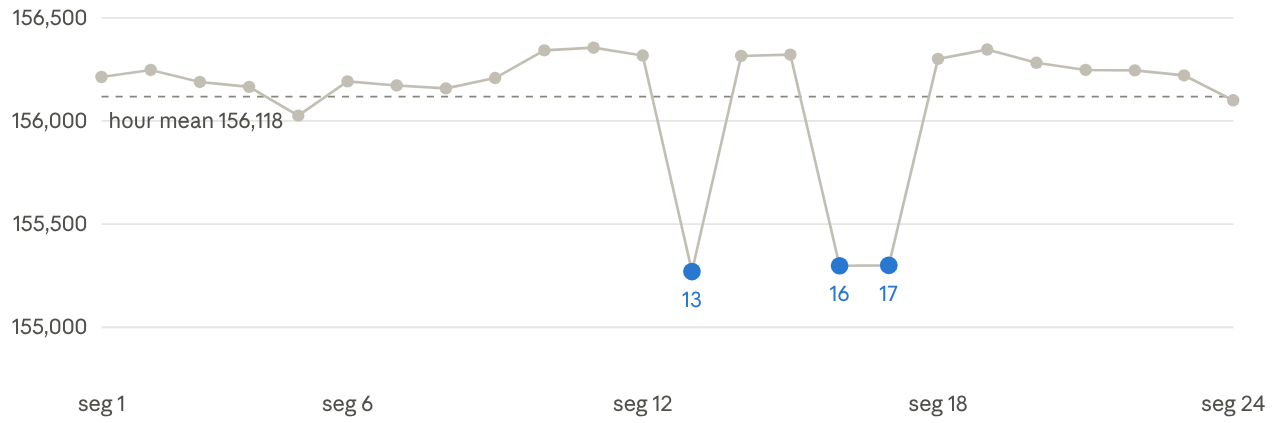
Conduit	Function codes	Registers	Rate	Poll interval
SCADA (10.10.2.10) to PLC	FC1 read, then 16 FC3 block reads	Coils 880 to 921; the status, photoeye, fault, and journal blocks	8.62 cycles/s	115.99 ms
SCADA to each VFD (10.10.1.21 to 26)	FC3 read	0 to 8	8.62 requests/s each	115.99 ms

A cycle is one full poll of a device, which can be several requests. SCADA's cycle is one FC1 read and 16 FC3 block reads, seen in all 24 segments and matching the OPC UA server source. Modbus TCP ADU sizes, excluding TCP/IP and Ethernet headers, are small and fixed: FC1 and FC3 requests are always 12 bytes, FC16 requests are 15 to 21 bytes depending on how many registers they write, FC16 responses are a 12-byte echo, and FC3 responses run 11 to 129 bytes.

Total traffic: `fw-l2` averaged about 524 packets per second and `plc-l1` about 1,040. The PLC link carries roughly double because it sees the routed SCADA and XLe traffic plus the local PLC-to-device polling that never reaches the firewall. No TCP connection closes or resets were observed within the captured windows and filters.

PLC link traffic held steady all hour; three segments dipped about 0.6%

plc-l1 packets per 150-second segment, 17:26 to 18:26 UTC



Segments 13, 16, 17: about 1,000 fewer packets; no kernel drops, resets, or exceptions

Packet counts from the 24 sealed segment records, plc-l1 tap, October 28, 2026

From the first segment to the last, the rate on both taps changed by less than 0.1%. Each loop's mean interval stayed within a 0.25% band across all 24 segments.

Normal behavior that could look like an attack

These are the parts of healthy idle most likely to trip a naive detection in Milestone 4. Each is normal for this range and has to be built into the rules.

- **Clockwork polling.** Each loop's mean polling interval stayed stable across all 24 segments. Beaconing detectors look for exactly this pattern in command-and-control traffic. Here it is just how industrial polling works, so detections must expect a fixed period per talker pair.
- **SCADA reading the VFDs directly across the firewall.** Someone expecting SCADA to talk only to the PLC would read this as a scope violation. It is by design: the firewall's `scada -> drives modbus` rule from Milestone 1 lets the HMI show drive status.
- **Constant Modbus writes.** The PLC writes to every VFD and scanner about 9.5 times a second, and the plant-profile traffic writes to the PLC about 10 times a second, roughly 95 writes per second at idle. A rule that fires on any write would fire constantly. The useful question for Milestone 4 is which source writes which registers.
- **A shared source address.** Plant-profile traffic comes from 10.10.1.21, which is also the address of the first VFD. A rule keyed only on source IP would blame the VFD for these writes. I attributed it to the plant process from its register profile alone; no process evidence confirms that.
- **Small rate dips.** Three segments ran about 0.6% below the rest with no reported kernel drops, resets, or Modbus exceptions. A tight rate threshold would alarm on them.

What I learned

- **An unrotated log can take down a whole system.** The VFD emulators wrote an idle status line every poll cycle for weeks until the root filesystem filled. Nothing failed loudly; the VM just stopped being usable. Log rotation is now a prerequisite before any longer capture.
- **A disk can be repaired without booting the machine.** I attached the stopped VM's disk image to the host with `qemu-nbd`, activated its LVM volume, and mounted it read-only to hash the logs. Then I remounted it read-write to export and truncate them.

Hashing first and truncating last meant the one irreversible step happened only after the archive was proven good. When the nbd device wedged, a host reboot was the clean fix.

- **Configuration that describes a running system goes stale.** Libvirt assigns vnet interface numbers in VM start order. After a reboot, the capture config pointed the PLC tap at the firewall's Level 3 interface. Checking each tap by MAC address before capturing caught it.
- **Idle is not quiet.** I expected a near-silent network with nothing sorting. Instead, four polling loops produce about 1,040 packets a second on the PLC link, all of it steady Modbus reads and writes. For Milestone 4, "is there Modbus write traffic" is a useless question. The useful one is which source writes which registers, at what rate.
- **A tool's limits shape the method.** The capture tool's 200,000-packet cap forced 150-second segments, and segmenting created small gaps I have to disclose. Knowing a tool's limits is part of the measurement.
- **AI agents need a human for judgment calls.** My agents spent two days on the disk problem, layering safety checks onto a backup that kept failing and waiting on approvals that never reached me. The fix took under an hour once I made the decisions and ran the root-level steps myself. The agents were most useful for read-only analysis and review, and a second agent's review caught overstated claims in the first draft of this report.

Limitations and next steps

This baseline covers one state from two vantage points. What it does not show:

- **One state only.** Normal sorting, recirculation, backpressure, drive stop/resume, and stopped recovery were not run.
- **Loopback traffic is unmeasured.** The OPC UA HMI subscription runs on `scada`'s loopback and XLe-to-ASX runs on `xle`'s loopback, where host taps cannot see. Port 8089 was also outside both filters, so this capture says nothing about it. The `drives` loopback capture is blocked in the capture config.
- **Counts are not paired.** The analyzer counts requests and responses separately; it does not match transaction IDs or measure latency. In segment 21, `plc-l1` has one more FC16 request than responses. One possible cause is a request at the segment's cutoff, but the evidence does not prove that.

- **One clock.** All timestamps come from the host clock. No offset between the VMs' clocks was measured, and there are about 1.3 seconds of unrecorded time between segments.

For Milestone 3, in order:

1. Add log rotation to the VFD emulators.
2. Capture the remaining operating states, with longer runs.
3. Add guest-side loopback captures for OPC UA and XLe-to-ASX.
4. Run the Milestone 3 attacks and measure each one against this baseline.

AI use. I used AI agents for capture and analysis tooling, for a first draft of the traffic analysis, and for a cross-model review that corrected two overstated claims. I ran the disk recovery, the range startup, and the capture myself, and checked the results.

Appendix: evidence index

All evidence paths are on the range host. Per-segment windows and packet counts are in the detailed analysis file. Full SHA-256 hashes for each pcap are in that segment's own SHA256SUMS manifest.

Item	Location
Sealed capture, 24 segments	`/home/kevin/vm/sorter-evidence/m2-healthy-idle-1h-20261028/seg01` to `seg24`
Readable analysis copy	`/home/kevin/vm/m2-analysis-copy/`
First test run, 193 s, hit the packet cap	`/home/kevin/vm/sorter-evidence/m2-healthy-idle-seg1-20261028/`
Operator startup and preflight notes (not yet filed)	`/home/kevin/vm/sorter-evidence/m2-healthy-idle-1h-20261028-operator-startup.md`
`drives` VFD log export, 6 files + SHA256SUMS	`/home/kevin/vm/sorter-evidence/m2-drives-vfd-logs-export-20261028T164317Z/`
Detailed traffic analysis	`MILESTONE2_BASELINE_HEALTHY_IDLE.md` in the repo

Item	Location
Code and tooling	`github.com/berryk6490/ics-sorter-range`, branch `milestone2/host-baseline-preparation`

Totals: `fw-l2` 1,888,482 packets, `plc-l1` 3,746,830 packets, zero reported kernel drops on either tap; interface-drop statistics were unavailable.